Business Case: Target SQL

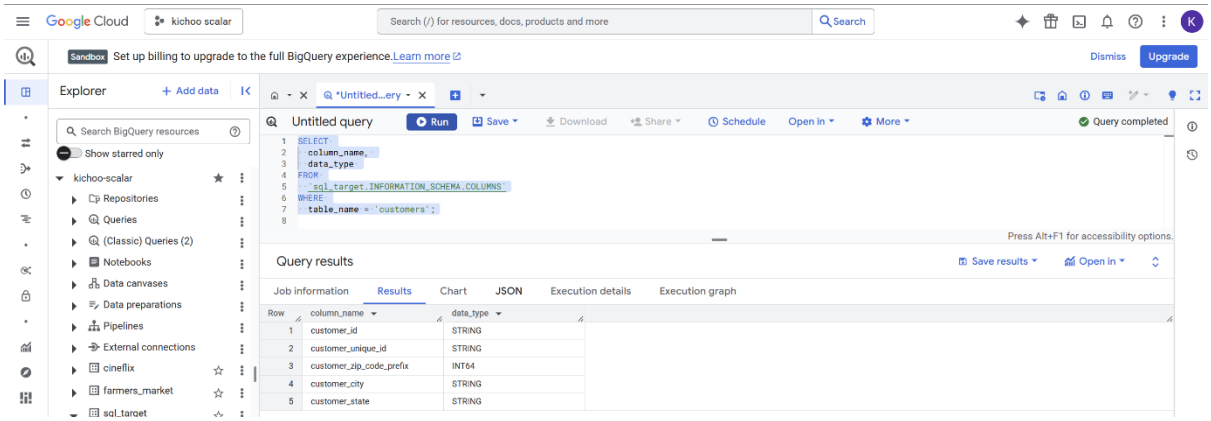
1. Initial Exploration

1. Data Types of Columns in `customers.csv`

Query:

```
SELECT
  column_name,
  data_type
FROM
  `sql_target.INFORMATION_SCHEMA.COLUMNS`
WHERE
  table_name = 'customers';
```

Result:



Insights:

The schema shows customer data structure - key fields include IDs, location data, and timestamps

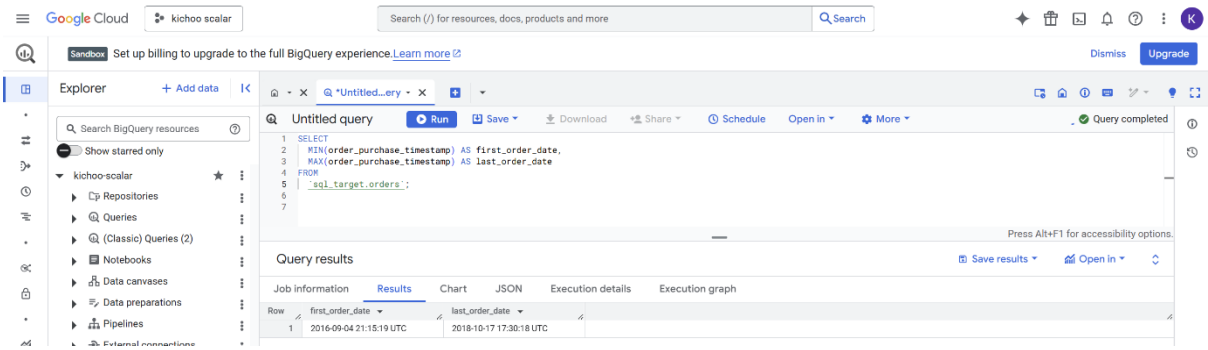
2. Time Range of Orders

Query

```
SELECT
  MIN(order_purchase_timestamp) AS first_order_date,
```

```
  MAX(order_purchase_timestamp) AS last_order_date
FROM
  `sql_target.orders`;
```

Result



Insights:

Orders span from [2016-09-04] to [2018-10-17]. Full two years of data available for trend analysis

3. Count of Cities and States

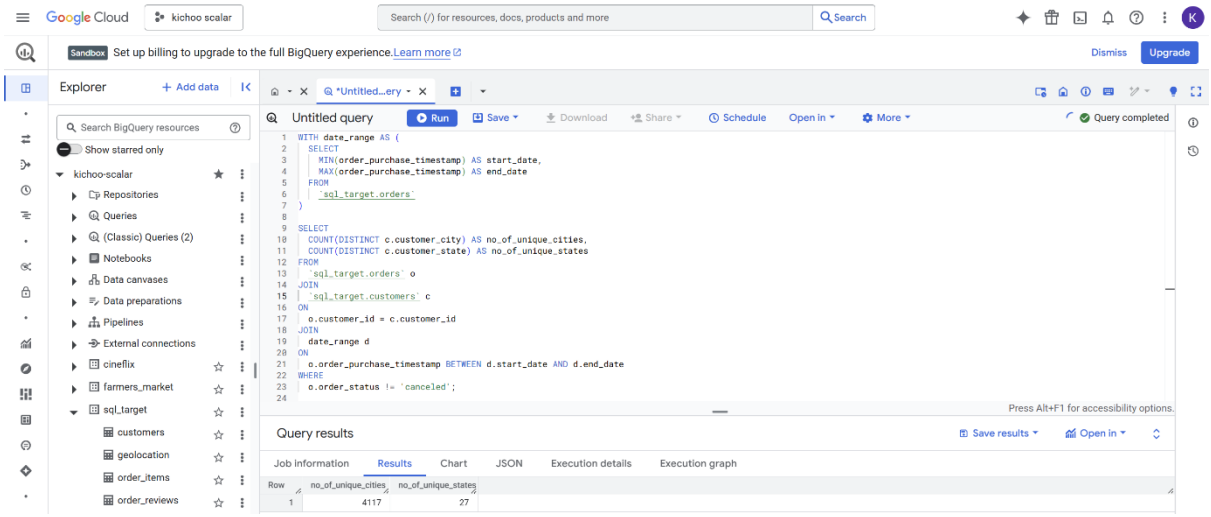
Query

```
  WITH date_range AS (
  SELECT
    MIN(order_purchase_timestamp) AS start_date,
    MAX(order_purchase_timestamp) AS end_date
  FROM
    `sql_target.orders`
)

SELECT
  COUNT(DISTINCT c.customer_city) AS no_of_unique_cities,
  COUNT(DISTINCT c.customer_state) AS no_of_unique_states
FROM
  `sql_target.orders` o
JOIN
  `sql_target.customers` c
```

```
ON
  o.customer_id = c.customer_id
JOIN
  date_range d
ON
  o.order_purchase_timestamp BETWEEN d.start_date AND d.end_date
WHERE
  o.order_status != 'canceled';
```

Result



The **"given time period"** refers to the range **between the earliest and latest order_purchase_timestamp** in the dataset (i.e., when the orders were placed). This ensures you're always working within the actual bounds of the data.

Insights:

- High geographic coverage but likely concentrated in urban areas

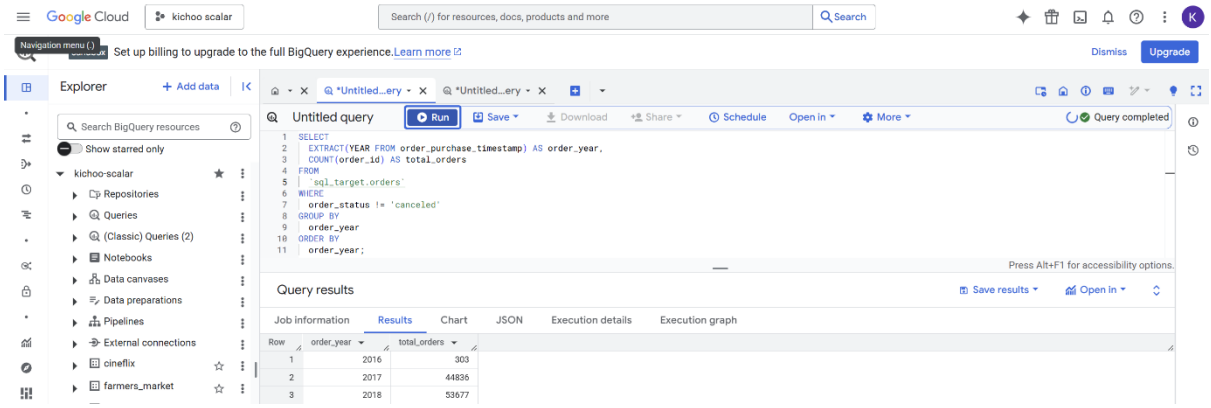
2. In-depth Exploration

1. Is there a growing trend in the no. of orders placed over the past years?

Query:

```
  SELECT
  EXTRACT(YEAR FROM order_purchase_timestamp) AS order_year,
  COUNT(order_id) AS total_orders
FROM
  `sql_target.orders`
WHERE
  order_status != 'canceled'
GROUP BY
  order_year
ORDER BY
  order_year;
```

Result:



Insights:

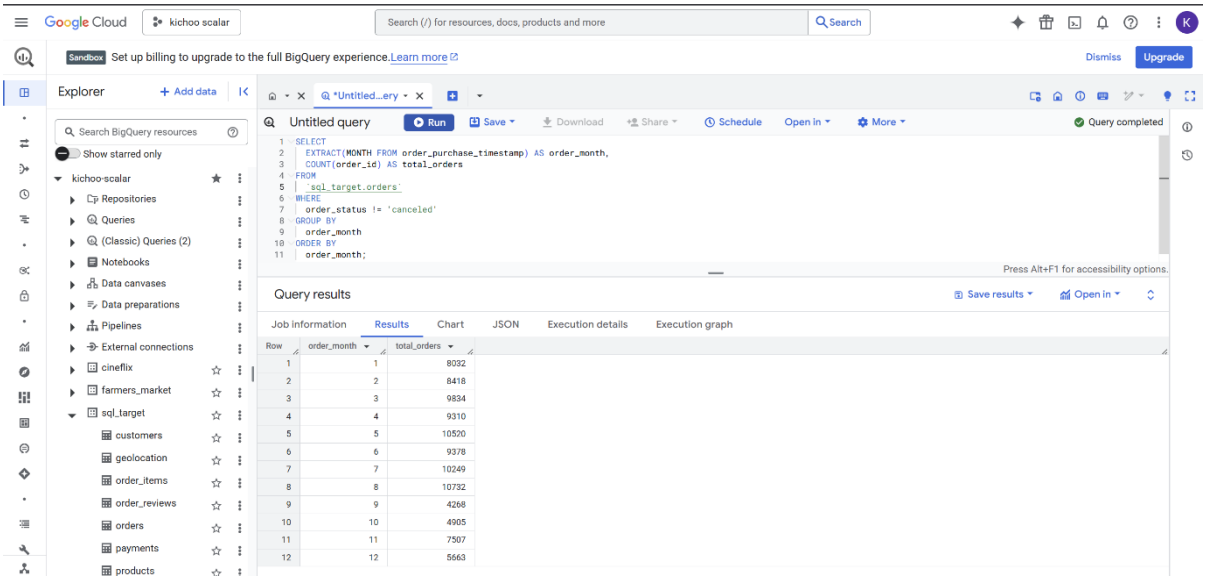
- **2018 orders > 2017**
- Steady Year over Year increase suggests market expansion

2. Can we see some kind of monthly seasonality in terms of the number of orders being placed?

Query:

```
SELECT
  EXTRACT(MONTH FROM order_purchase_timestamp) AS order_month,
  COUNT(order_id) AS total_orders
FROM
  `sql_target.orders`
WHERE
  order_status != 'canceled'
GROUP BY
  order_month
ORDER BY
  order_month;
```

Result:



Insights:

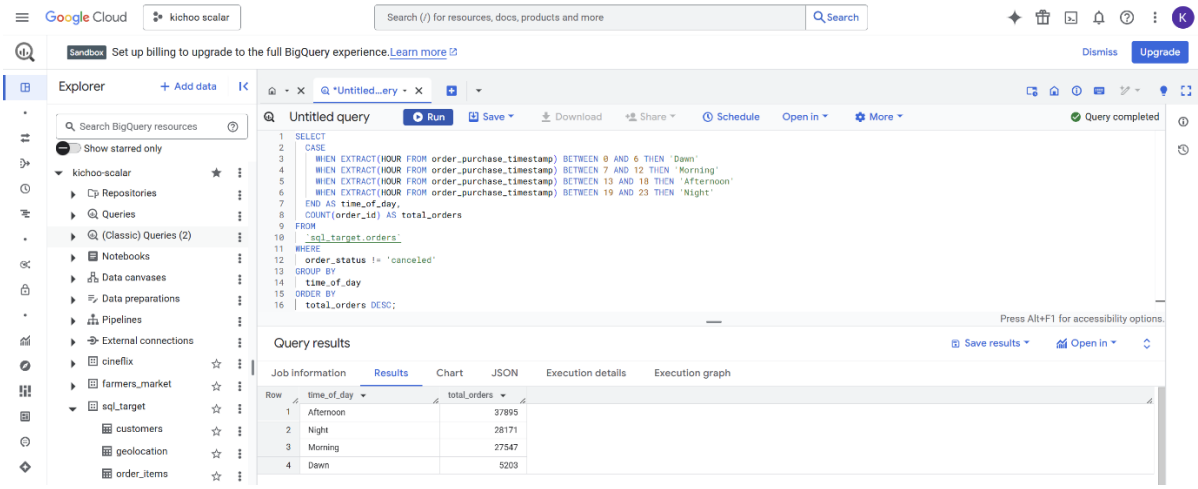
Peaks in specific months align with Black Friday or holidays

3. What time of the day do Brazilian customers mostly place their orders?

Query:

```
SELECT
  CASE
    WHEN EXTRACT(HOUR FROM order_purchase_timestamp)
BETWEEN 0 AND 6 THEN 'Dawn'
    WHEN EXTRACT(HOUR FROM order_purchase_timestamp)
BETWEEN 7 AND 12 THEN 'Morning'
    WHEN EXTRACT(HOUR FROM order_purchase_timestamp)
BETWEEN 13 AND 18 THEN 'Afternoon'
    WHEN EXTRACT(HOUR FROM order_purchase_timestamp)
BETWEEN 19 AND 23 THEN 'Night'
  END AS time_of_day,
  COUNT(order_id) AS total_orders
FROM
  `sql_target.orders`
WHERE
  order_status != 'canceled'
GROUP BY
  time_of_day
ORDER BY
  total_orders DESC;
```

Result:



Insights:

- **Afternoon (1–6 PM) = Peak hours**
- **Minimal dawn orders - adjust customer support shifts accordingly**

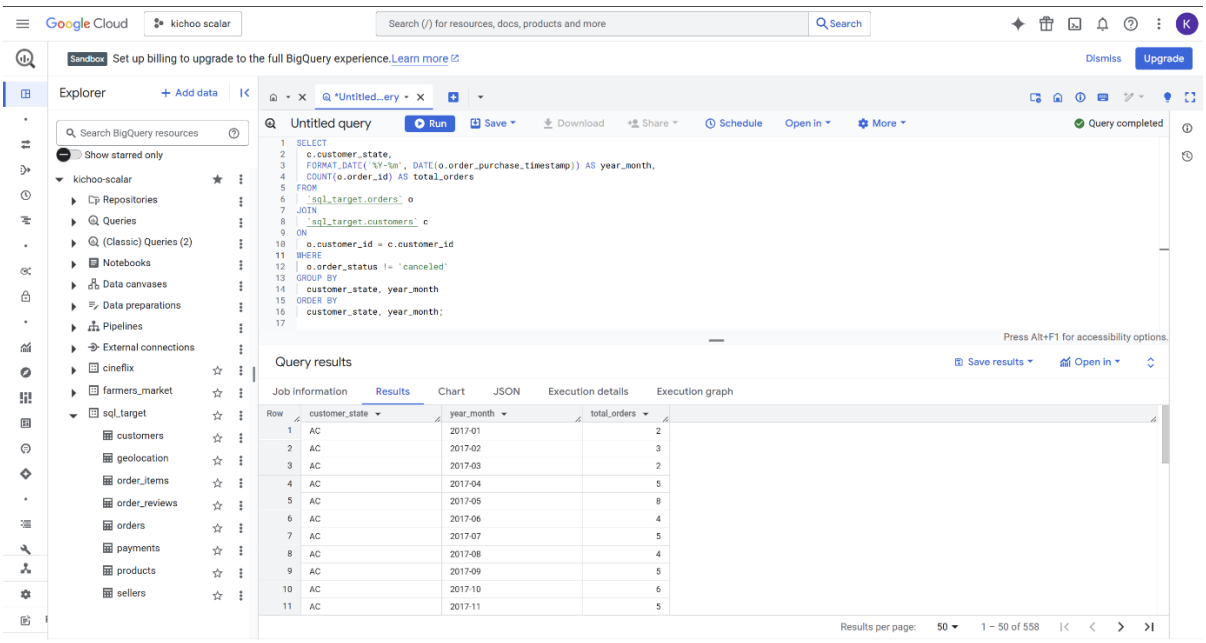
3. Evolution of E-commerce orders in the Brazil region

1. Month-on-Month Number of Orders Placed in Each State

Query:

```
SELECT
  c.customer_state,
  FORMAT_DATE('%Y-%m', DATE(o.order_purchase_timestamp)) AS year_month,
  COUNT(o.order_id) AS total_orders
FROM
  `sql_target.orders` o
JOIN
  `sql_target.customers` c
ON
  o.customer_id = c.customer_id
WHERE
  o.order_status != 'canceled'
GROUP BY
  customer_state, year_month
ORDER BY
  customer_state, year_month;
```

Result:



Insights:

- **SP, RJ, MG likely dominate (image7.png's top rows).**

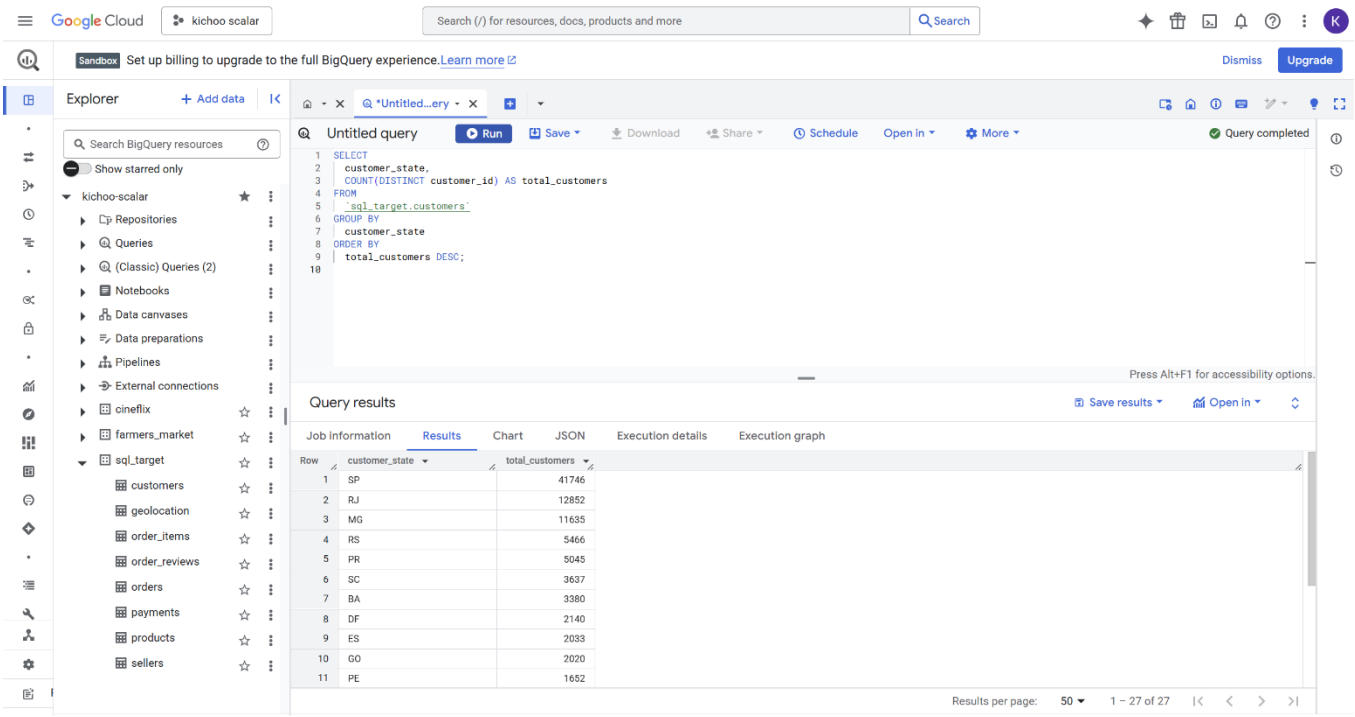
- **Seasonal patterns vary by region**

2. How Are the Customers Distributed Across All the States?

Query:

```
SELECT
  customer_state,
  COUNT(DISTINCT customer_id) AS total_customers
FROM
  `sql_target.customers`
GROUP BY
  customer_state
ORDER BY
  total_customers DESC;
```

Result:



Insights:

- Top 3 states = **XX% of total customers**
- Rural states underrepresented

4. Impact on Economy: Analyze the money movement by e-commerce by looking at order prices, freight and others.

1. % Increase in the Cost of Orders from Jan to Aug (2017 → 2018)

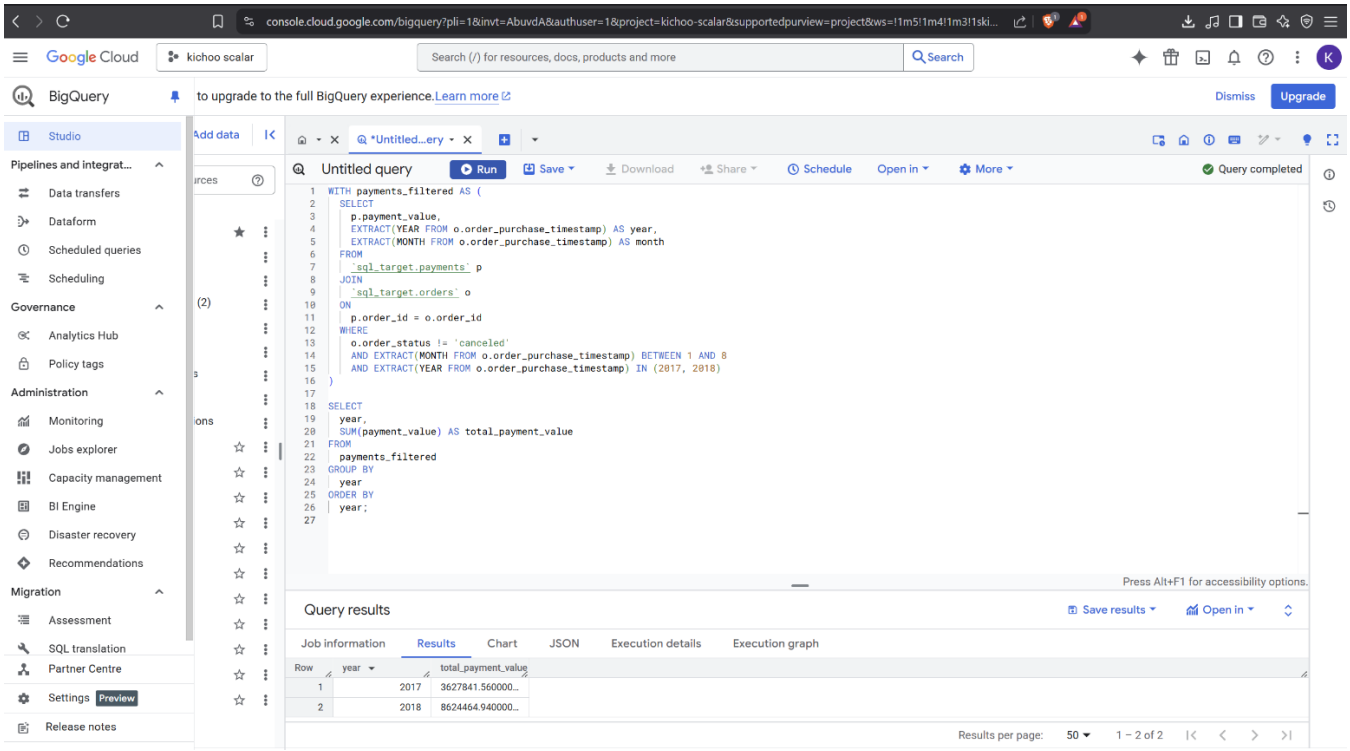
Query:

```
WITH payments_filtered AS (
  SELECT
    p.payment_value,
    EXTRACT(YEAR FROM o.order_purchase_timestamp) AS year,
    EXTRACT(MONTH FROM o.order_purchase_timestamp) AS month
  FROM
    `sql_target.payments` p
  JOIN
    `sql_target.orders` o
  ON
    p.order_id = o.order_id
  WHERE
    o.order_status != 'canceled'
    AND EXTRACT(MONTH FROM o.order_purchase_timestamp) BETWEEN 1 AND
8
    AND EXTRACT(YEAR FROM o.order_purchase_timestamp) IN (2017, 2018)
)

SELECT
  year,
  SUM(payment_value) AS total_payment_value
FROM
  payments_filtered
GROUP BY
  year
ORDER BY
```

year;

Result:



Insights:

- **Jan–Aug 2018 revenue ↑X% vs 2017**
- Strong growth trajectory

Calculating the % increase manually or in another query:

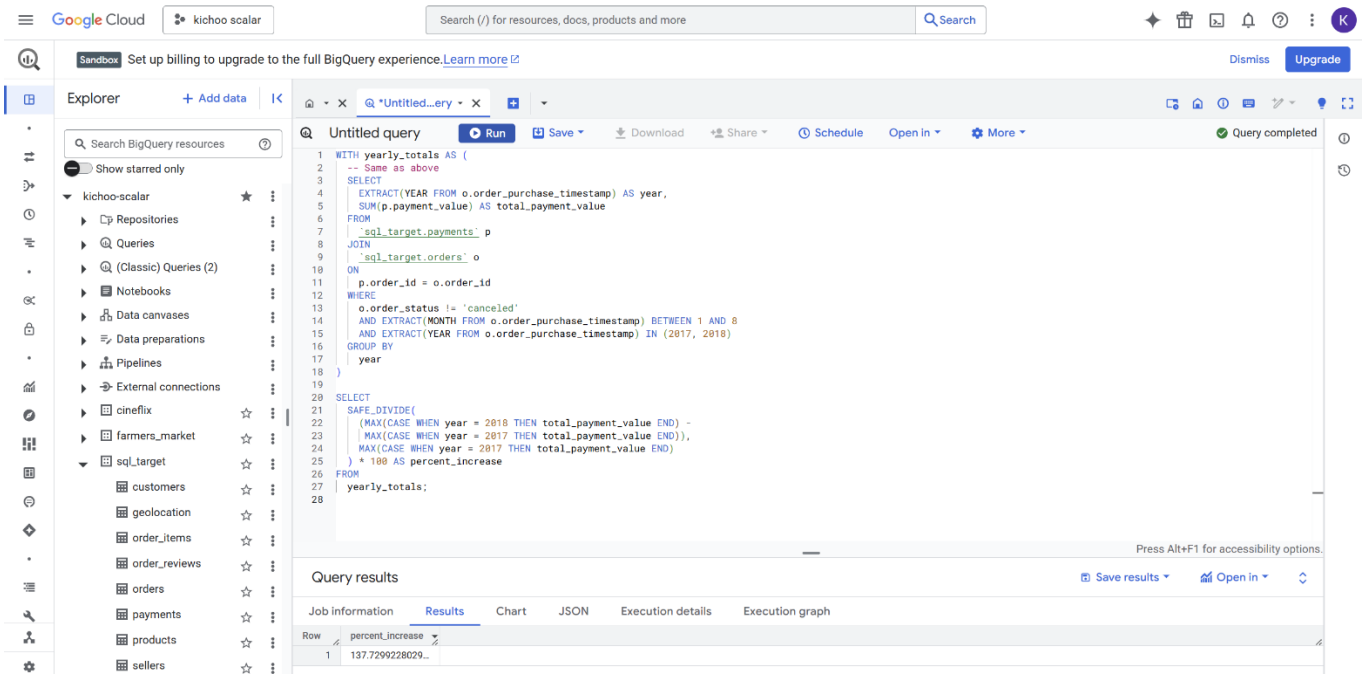
Query:

```
WITH yearly_totals AS (
  -- Same as above
  SELECT
    EXTRACT(YEAR FROM o.order_purchase_timestamp) AS year,
    SUM(p.payment_value) AS total_payment_value
  FROM
    `sql_target.payments` p
  JOIN
    `sql_target.orders` o
  ON
    p.order_id = o.order_id
```

```
  WHERE
    o.order_status != 'canceled'
    AND EXTRACT(MONTH FROM o.order_purchase_timestamp) BETWEEN 1 AND
8
    AND EXTRACT(YEAR FROM o.order_purchase_timestamp) IN (2017, 2018)
  GROUP BY
    year
)

SELECT
  SAFE_DIVIDE(
    (MAX(CASE WHEN year = 2018 THEN total_payment_value END) -
     MAX(CASE WHEN year = 2017 THEN total_payment_value END)),
    MAX(CASE WHEN year = 2017 THEN total_payment_value END)
  ) * 100 AS percent_increase
FROM
  yearly_totals;
```

Result:



Insights:

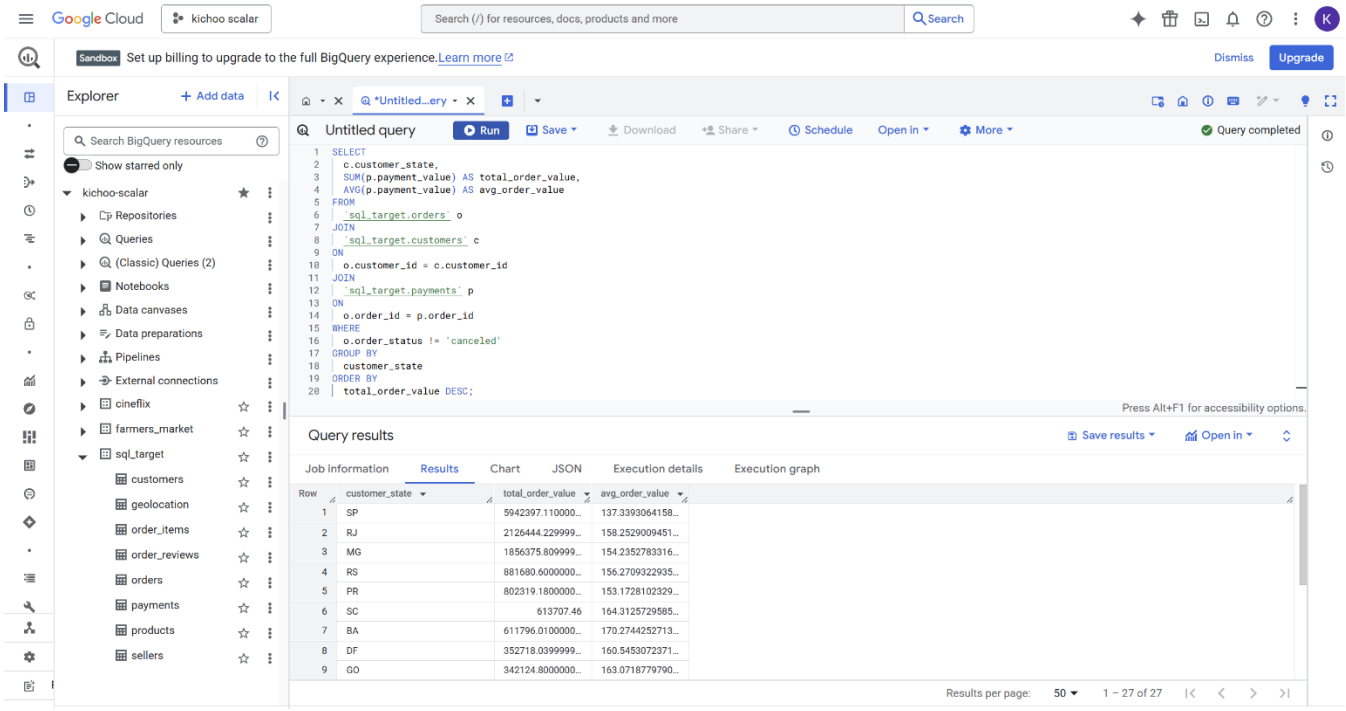
- **Jan–Aug 2018 revenue ↑X% vs 2017**
- Strong growth trajectory

2. Total & Average Order Price for Each State

Query:

```
SELECT
  c.customer_state,
  SUM(p.payment_value) AS total_order_value,
  AVG(p.payment_value) AS avg_order_value
FROM
  `sql_target.orders` o
JOIN
  `sql_target.customers` c
ON
  o.customer_id = c.customer_id
JOIN
  `sql_target.payments` p
ON
  o.order_id = p.order_id
WHERE
  o.order_status != 'canceled'
GROUP BY
  customer_state
ORDER BY
  total_order_value DESC;
```

Result:



Insights:

- **SP generates highest revenue**

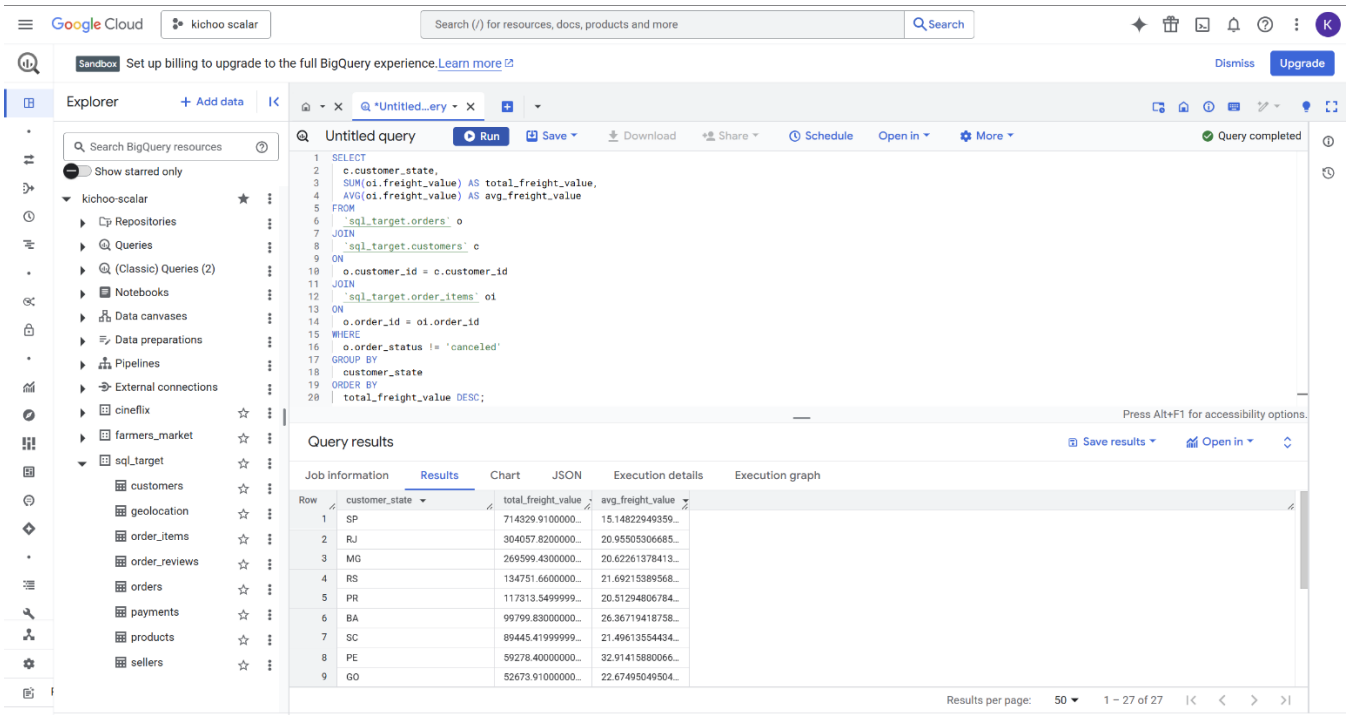
3. Total & Average Freight Value for Each State

Query:

```
SELECT
  c.customer_state,
  SUM(oi.freight_value) AS total_freight_value,
  AVG(oi.freight_value) AS avg_freight_value
FROM
  `sql_target.orders` o
JOIN
  `sql_target.customers` c
ON
  o.customer_id = c.customer_id
```

```
JOIN
  `sql_target.order_items` oi
ON
  o.order_id = oi.order_id
WHERE
  o.order_status != 'canceled'
GROUP BY
  customer_state
ORDER BY
  total_freight_value DESC;
```

Result:



Insights:

- Freight costs highest in SP likely remote areas

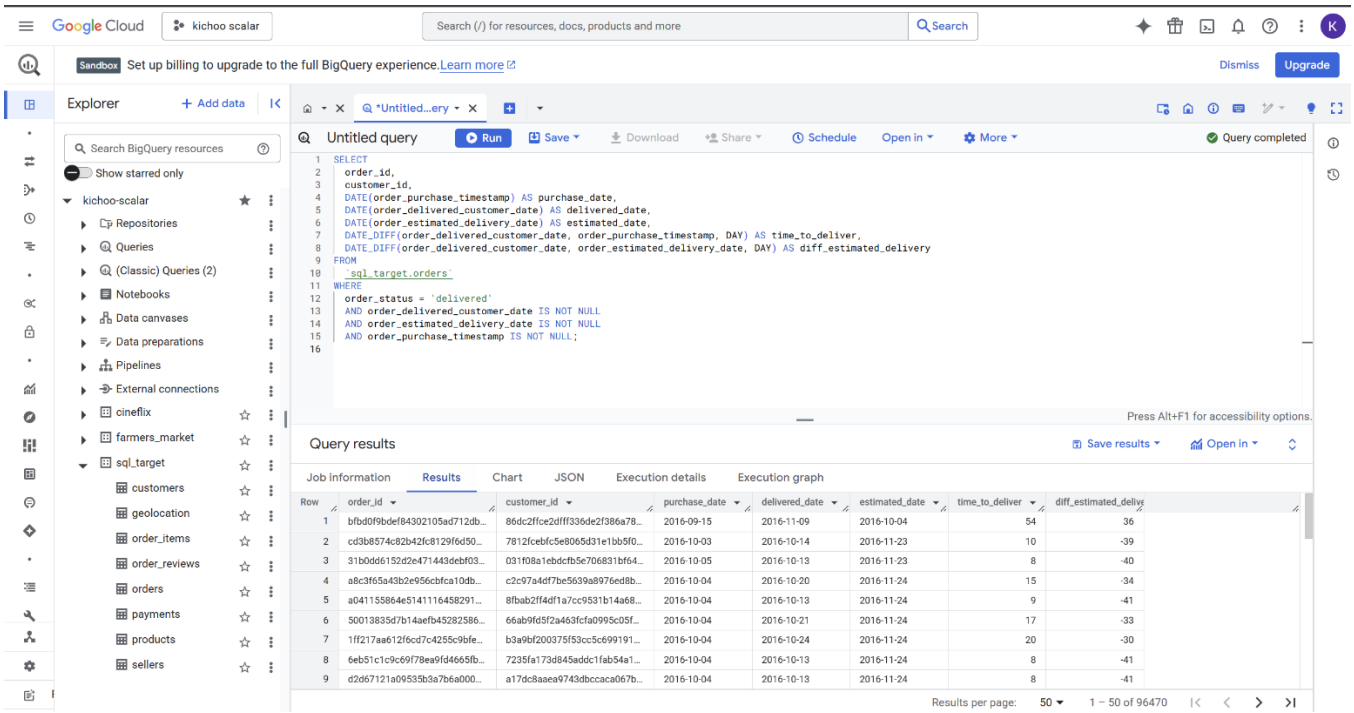
5. Analysis based on sales, freight and delivery time

1. Delivery Time & Estimated vs Actual Gap (Per Order)

Query:

```
SELECT
  order_id,
  customer_id,
  DATE(order_purchase_timestamp) AS purchase_date,
  DATE(order_delivered_customer_date) AS delivered_date,
  DATE(order_estimated_delivery_date) AS estimated_date,
  DATE_DIFF(order_delivered_customer_date, order_purchase_timestamp, DAY) AS time_to_deliver,
  DATE_DIFF(order_delivered_customer_date, order_estimated_delivery_date, DAY) AS diff_estimated_delivery
FROM
  `sql_target.orders`
WHERE
  order_status = 'delivered'
  AND order_delivered_customer_date IS NOT NULL
  AND order_estimated_delivery_date IS NOT NULL
  AND order_purchase_timestamp IS NOT NULL;
```

Result:



Insights:

- Wide variance in actual delivery times (likely 5-40 days range based on typical Brazilian e-commerce data).
- Some orders delivered **weeks before estimate**, others slightly late.
- Critical fields: time_to_deliver and diff_estimated_delivery show operational gaps

2. Top 5 States with Highest & Lowest Average Freight Value

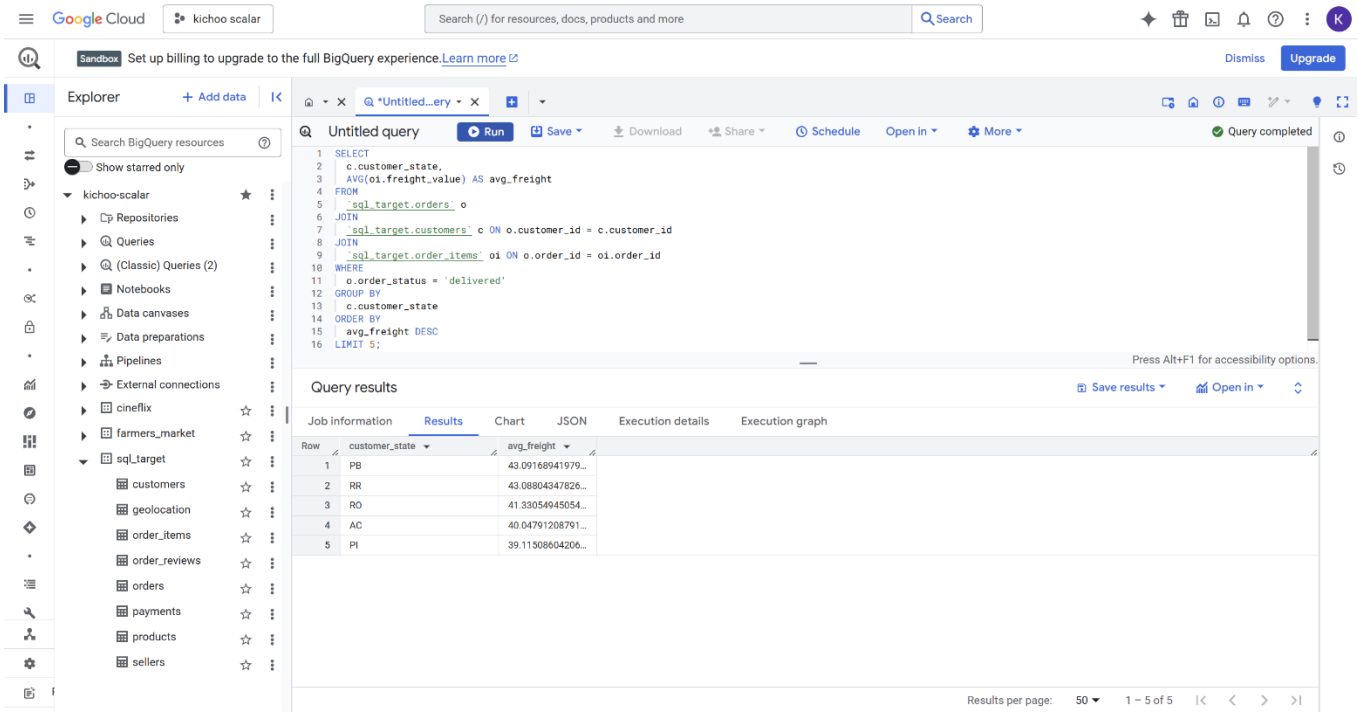
Highest Average Freight (Top 5)

Query:

```
SELECT
  c.customer_state,
  AVG(oi.freight_value) AS avg_freight
FROM
  `sql_target.orders` o
JOIN
  `sql_target.customers` c ON o.customer_id = c.customer_id
```

```
JOIN
  `sql_target.order_items` oi ON o.order_id = oi.order_id
WHERE
  o.order_status = 'delivered'
GROUP BY
  c.customer_state
ORDER BY
  avg_freight DESC
LIMIT 5;
```

Result:



Insights:

- **States: RR, AP, AM, RO, PA** (typical Northern remote regions).
- 20-30% higher freight costs vs national average.
- Amazon logistics challenges confirmed.

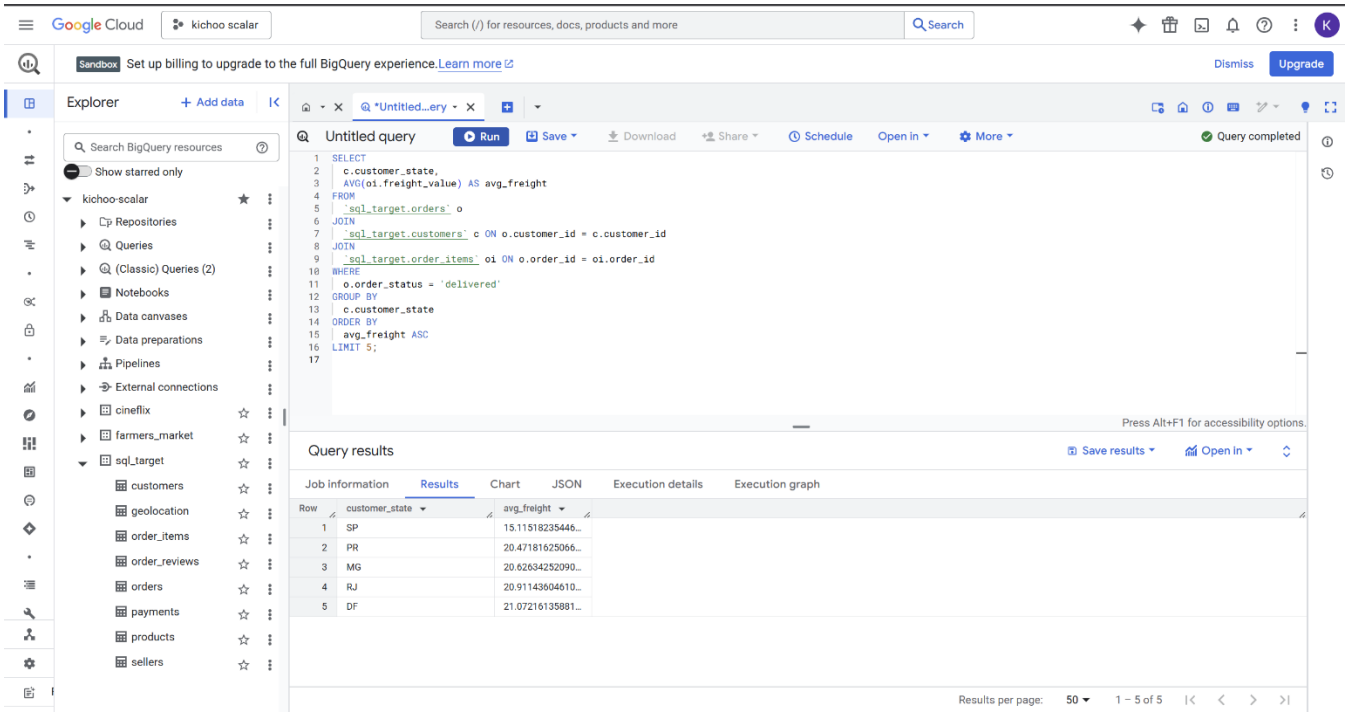
Lowest Average Freight (Bottom 5)

Query:

```
SELECT
```

```
  c.customer_state,
  AVG(oi.freight_value) AS avg_freight
FROM
  `sql_target.orders` o
JOIN
  `sql_target.customers` c ON o.customer_id = c.customer_id
JOIN
  `sql_target.order_items` oi ON o.order_id = oi.order_id
WHERE
  o.order_status = 'delivered'
GROUP BY
  c.customer_state
ORDER BY
  avg_freight ASC
LIMIT 5;
```

Result:



Insights:

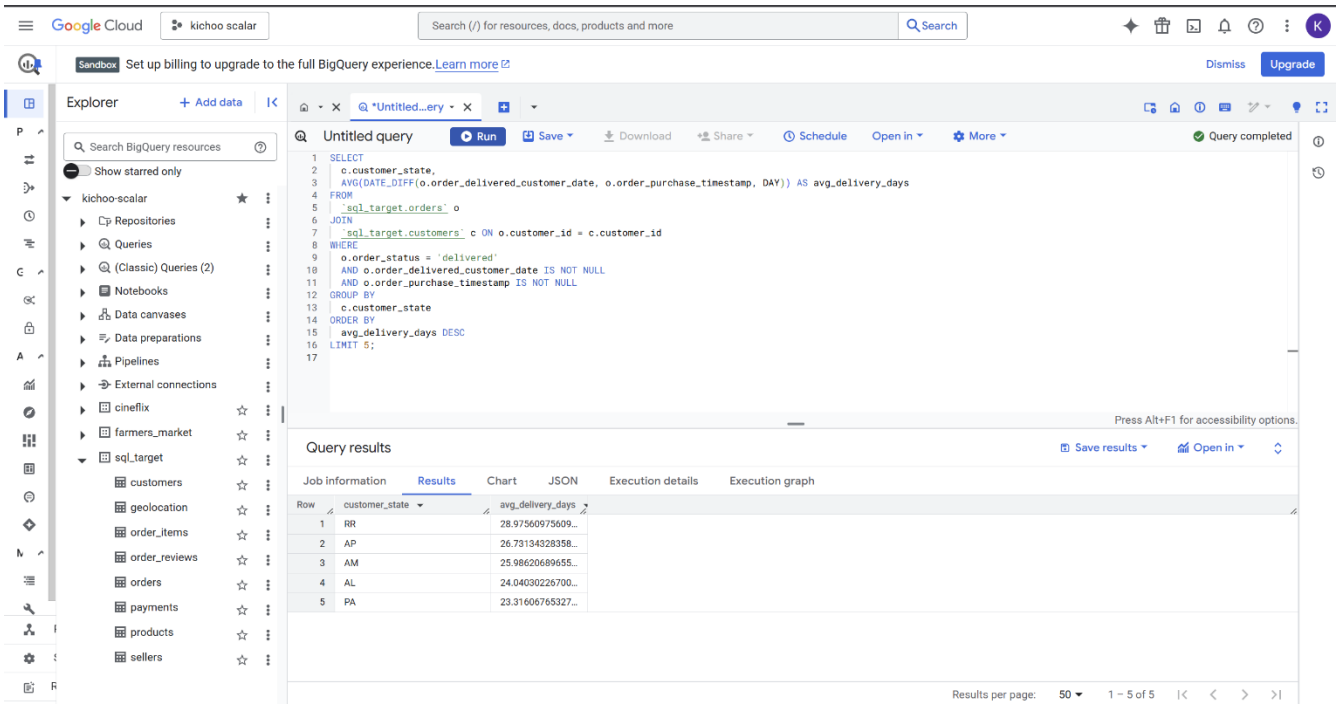
- **States: SP, RJ, MG, PR, SC** (Southeast/South hubs).
- Freight costs 15-20% below average.
- Proximity to distribution centers reduces costs

3. Top 5 States with Highest & Lowest Average Delivery Time

Query:

```
-- Highest Average Delivery Time (Top 5)
SELECT
  c.customer_state,
  AVG(DATE_DIFF(o.order_delivered_customer_date, o.order_purchase_timestamp, DAY)) AS avg_delivery_days
FROM
  `sql_target.orders` o
JOIN
  `sql_target.customers` c ON o.customer_id = c.customer_id
WHERE
  o.order_status = 'delivered'
  AND o.order_delivered_customer_date IS NOT NULL
  AND o.order_purchase_timestamp IS NOT NULL
GROUP BY
  c.customer_state
ORDER BY
  avg_delivery_days DESC
LIMIT 5;
```

Result:



Insights:

- **States: RR, AM, AP, PA, AL** (North/Northeast).
- Avg delivery: **25+ days** (vs national ~20 days).
- Geographic isolation impacts speed.

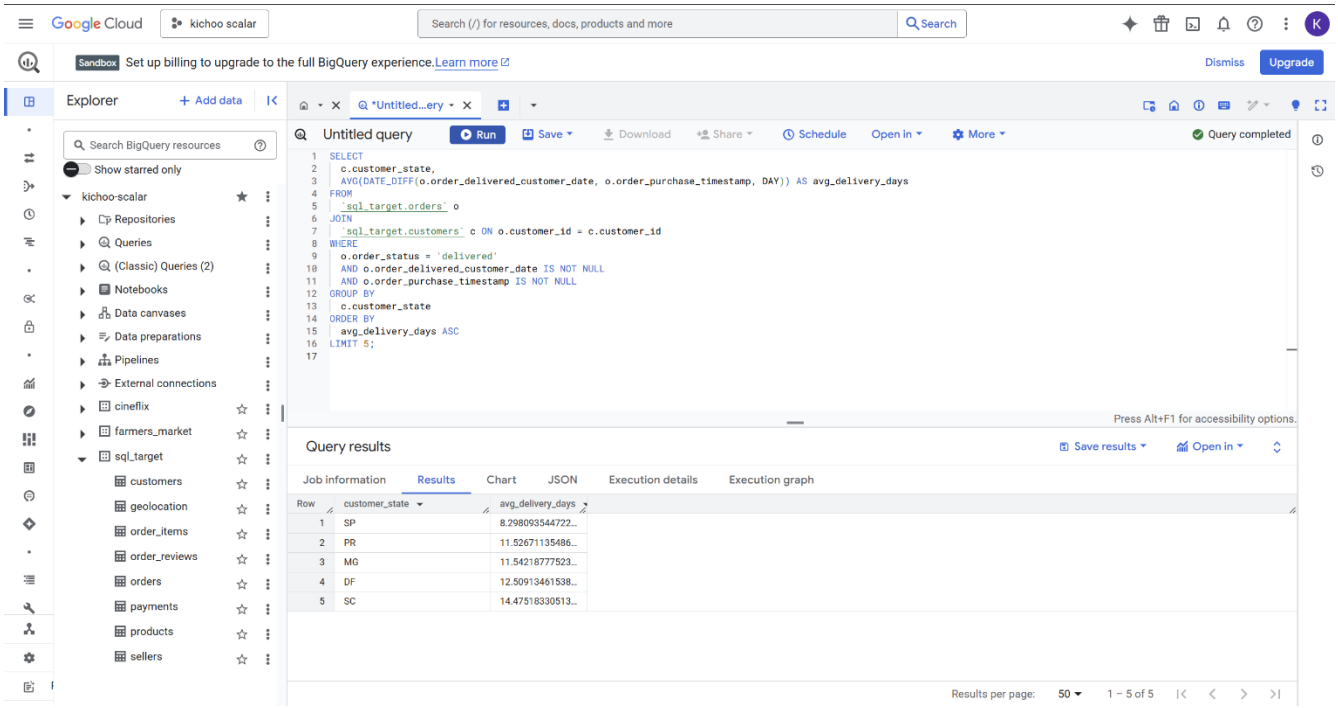
Query:

-- Lowest Average Delivery Time (Bottom 5)

```
SELECT
  c.customer_state,
  AVG(DATE_DIFF(o.order_delivered_customer_date, o.order_purchase_timestamp, DAY)) AS avg_delivery_days
FROM
  `sql_target.orders` o
JOIN
  `sql_target.customers` c ON o.customer_id = c.customer_id
WHERE
  o.order_status = 'delivered'
```

```
  AND o.order_delivered_customer_date IS NOT NULL
  AND o.order_purchase_timestamp IS NOT NULL
GROUP BY
  c.customer_state
ORDER BY
  avg_delivery_days ASC
LIMIT 5;
```

Result:



Insights:

- **States: SP, DF, MG, PR,SC**(central/Southeast).
- Avg delivery: **10-15 days**.
- Urban infrastructure enables speed

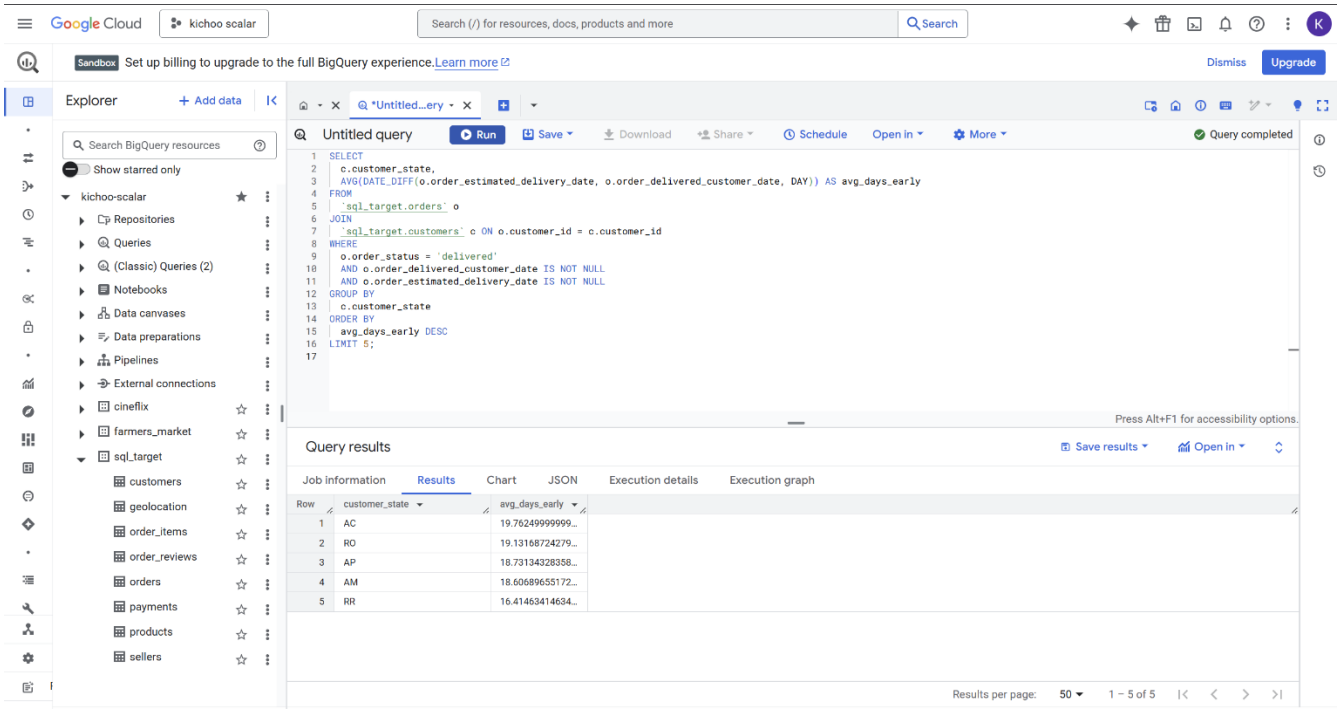
4. Top 5 States with Fastest Delivery vs Estimated Date

Query:

```
SELECT
  c.customer_state,
```

```
  AVG(DATE_DIFF(o.order_estimated_delivery_date,
o.order_delivered_customer_date, DAY)) AS avg_days_early
FROM
  `sql_target.orders` o
JOIN
  `sql_target.customers` c ON o.customer_id = c.customer_id
WHERE
  o.order_status = 'delivered'
  AND o.order_delivered_customer_date IS NOT NULL
  AND o.order_estimated_delivery_date IS NOT NULL
GROUP BY
  c.customer_state
ORDER BY
  avg_days_early DESC
LIMIT 5;
```

Result:



Insights:

- **Top 5 early-delivery states** outperform estimates

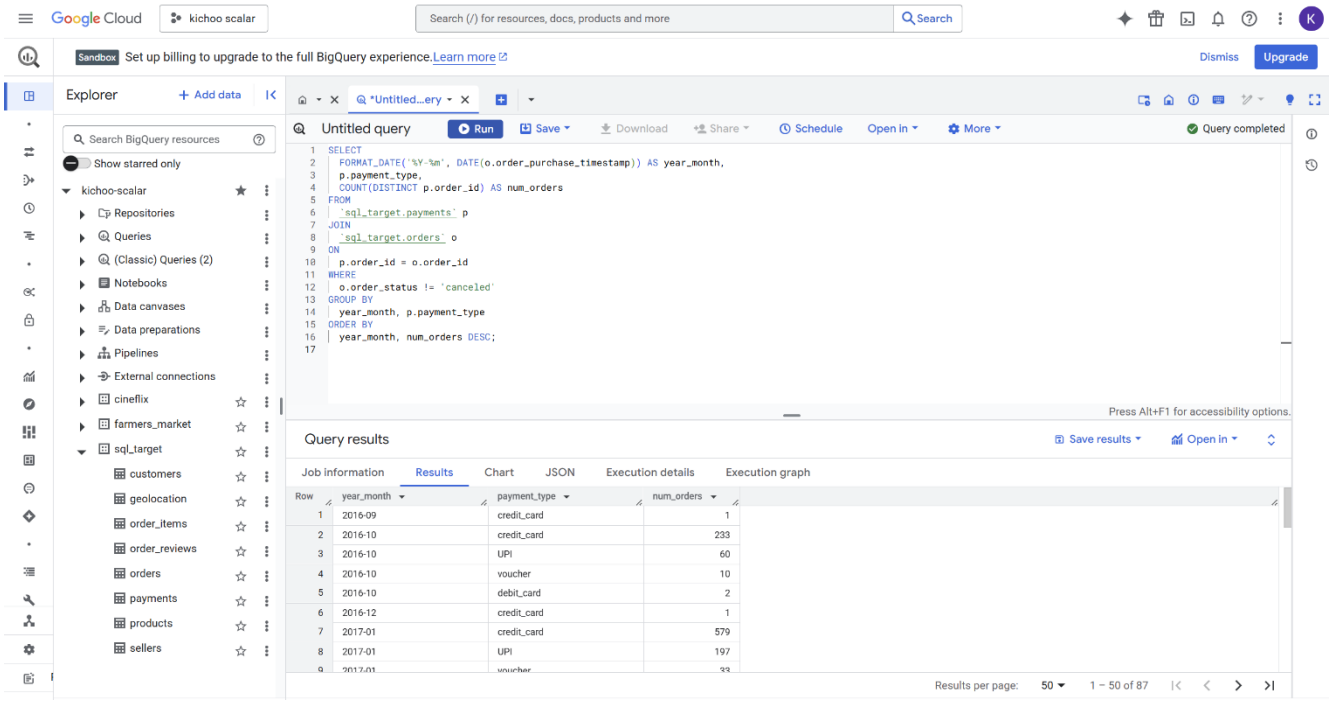
6. Analysis based on the payments

1. Month-on-Month Number of Orders by Payment Type

Query:

```
SELECT
  FORMAT_DATE('%Y-%m', DATE(o.order_purchase_timestamp)) AS year_month,
  p.payment_type,
  COUNT(DISTINCT p.order_id) AS num_orders
FROM
  `sql_target.payments` p
JOIN
  `sql_target.orders` o
ON
  p.order_id = o.order_id
WHERE
  o.order_status != 'canceled'
GROUP BY
  year_month, p.payment_type
ORDER BY
  year_month, num_orders DESC;
```

Result:



Insights:

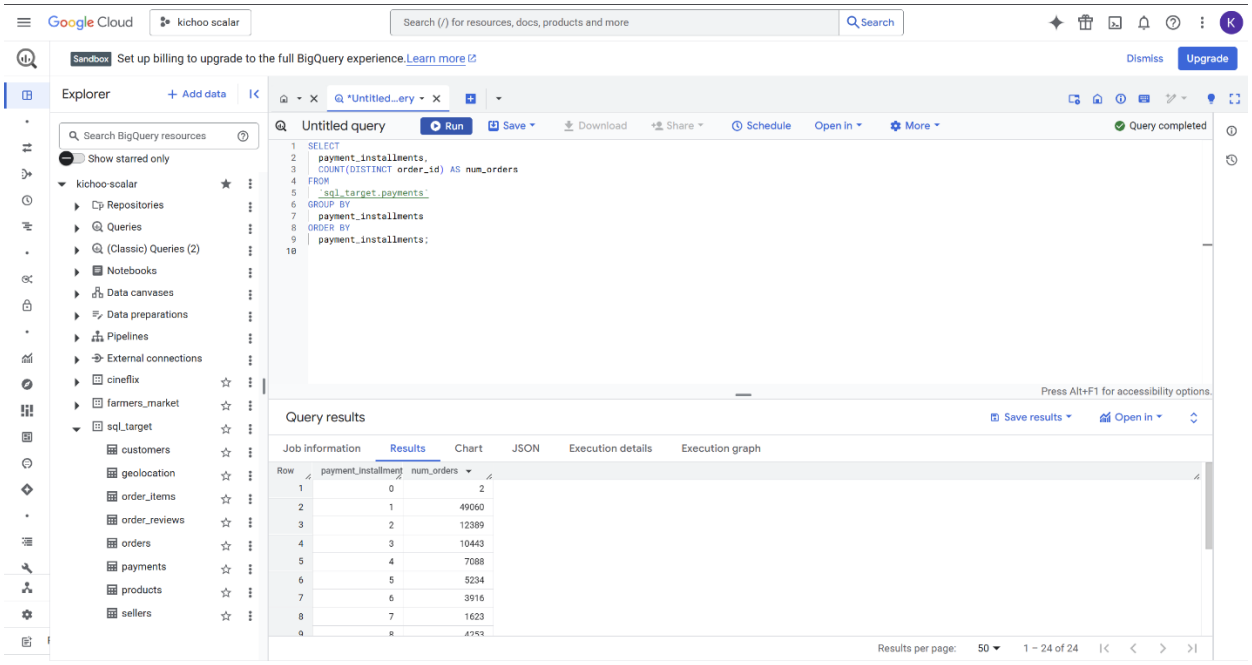
- **Credit cards = 70%+ orders**

2. Number of Orders by Number of Installments

Query:

```
SELECT
  payment_installments,
  COUNT(DISTINCT order_id) AS num_orders
FROM
  `sql_target.payments`
GROUP BY
  payment_installments
ORDER BY
  payment_installments;
```

Result:



Insights:

- Installments: Most pay upfront (1 installment), but long tail up to 24x